Lecture 4

ALU Instructions

ALU Instructions

- **ALU (Arithmetic Logic Unit)**: A critical component of the CPU, performing all arithmetic and logical operations.
- **Role in Computer Architecture**: Executes instructions that manipulate data, directly impacting processing speed and efficiency.

Types of ALU Instructions

- Arithmetic Instructions
  - Addition (ADD): Adds two operands.
  - Subtraction (SUB): Subtracts one operand from another.
  - Multiplication (MUL)
  - Division (DIV)
  - NEG (Negate): Changes the sign of an operand by multiplying with -1
  - ADC (Add with Carry)
  - SBB (Subtract with Borrow)

ADD

- Performs addition of two operands
- Operands can be registers, immediate values, or memory locations.
- ADD destination, source
- ADD AX, BX
- AX and BX will be added and result will be stored in AX

SUB

- Performs binary subtraction of one operand from another.
- Operands can be registers, immediate values, or memory locations.
- SUB destination, source
- SUB AX, BX
- Subtracts value in BX from that of AX and stores the result in AX

MUL

- Performs multiplication of two operands, yielding a product.
- Single-operand instruction (can’t be immediate value)
- Destination will always be AX along with DX (in case of overflow)
- DX will be used if result exceeds 16 bits.
- MUL source e.g. MUL CX
- Same as MUL AX, CX

DIV

- Performs division of two operands.
- The instruction divides an implicit dividend in AX by the specified operand (usually a register or memory location).
- DIV divisor
- DIV CX divides value of AX by CX
- Quotient is stored in AX and remainder in DX

ADC

- Performs addition of two operands, including the carry flag.
- ADC destination, source
- ADC AX, BX
  - If AX contains 10 and BX contains 5
  - Check for CF; is 1
  - After execution AX is 10+5+1 = 16

SBB

- Performs subtraction of two operands, considering the borrow flag from a previous operation.
- SBB destination, source
- SBB AX, BX
  - If AX contains 10 and BX contains 3
  - Check for CF; is 1
  - After execution AX is 10-3-1 = 6
- Carry flag acts as borrow flag

NEG

- Negates (inverts) the value of the specified operand, effectively performing multiplication by -1.
- NEG operand (NEG AX)
- Before execution AX is 5
- After execution AX is -5 (FFFB)

Types of ALU Instructions

- Logic Instructions
  - Perform bitwise logical operations on binary values.
  - AND
  - OR
  - NOT
  - XOR

AND

- Performs a bitwise AND operation.
- AND destination, source
- AND AX, BX
- AX = 0b1100 0011 and BX = 0b1010 1111
- After execution AX will be 0b1000 0011

OR

- Performs a bitwise OR operation.
- OR destination, source
- OR AX, BX
- AX = 0b1100 1010, BX = 0b1010 1111
- After execution AX will be AX = 0b1110 1111

XOR

- Performs a bitwise exclusive OR operation.
- XOR destination, source
- XOR AX, BX
- AX = 0b1100 1010, BX = 0b1010 1111
- After execution, AX = 0b0110 0101

NOT

- Performs a bitwise NOT operation (bitwise complement).
- Inverts all bits of the operand.
- NOT operand
- NOT AX
- AX = 0b1100 1010
- After execution, AX = 0b0011 0101

Flag Register

- 16-bit register in the Intel 8086 microprocessor
- Contains information about the state of the processor after executing an instruction
- Also known as status register
- The various flags in the flag register are set or cleared based on the result of arithmetic, logic, and other instructions executed by the processor.

Format of Flag Register

- Total 16 bits
  - 9 used by flags
  - 6 are unused to provide flexibility for future use (set to 0)
- 9 used bits
  - 6 status/conditional flags
  - 3 control flags

Status Flags

- Set to 1 or cleared to 0 after execution of each instruction
  - Sign flag (SF)
  - Zero flag (ZF)
  - Auxiliary carry flag (ACF)
  - Parity flag (PF)
  - Overflow flag (OF)
  - Carry flag (CF)

Sign Flag

- Indicates the sign of the result of the last arithmetic or logical operation
- It helps determine whether the result of an operation is positive or negative
- The Sign Flag is the most significant bit (MSB) of the result in binary representation.
- SF = 1; MSB = 1
- SF = 0; MSB = 0

Example

- AX = 0b1000 0001 (129) + 0b1000 0001 (129)
- AX = 0b0001 0000 (256)
- SF = 0 (positive result)
- AX = 0b0000 0001 (1) - 0b0000 0010 (2)
- AX = 0b1111 1111 (255) (in two's complement, represents -1)
- SF = 1 (negative result)

Zero Flag

- Indicates whether the result of the last arithmetic or logical operation was zero.
- ZF = 1: Indicates the result of the last operation is zero.
- ZF = 0: Indicates the result is non-zero.
- AX = 0b0000 0000 (0) + 0b0000 0000 (0)
  - AX = 0b0000 0000 (0); ZF is set to 1
- AX = 0b0000 0001 (1) + 0b0000 0001 (1)
  - AX = 0b0000 0010 (2); ZF is set to 0

Auxiliary Carry Flag

- This flag is used in BCD (Binary-coded Decimal) operations.
- Indicates whether a carry occurred from the lower nibble (the least significant 4 bits) to the upper nibble during an arithmetic operation.

- 29H = 0010 1001
- +4CH = 0100 1100
- 75H = 0111 0101
- AF is set to 1

0000 1001 (9)
+ 0000 0001 (1)

0000 1010 (10)

- AF is set to 0

Parity Flag

- Indicates whether the number of set bits (1s) in the result of the last arithmetic or logical operation is even or odd.
- PF = 1: Indicates an even number of 1s in the result (even parity).
- PF = 0: Indicates an odd number of 1s in the result (odd parity).

PF = 1

```
   1 1
  1011 1100
+ 1001 0001
-----------
1 0100 1101
```

PF = 0

```
   1 1
  1011 1100
+ 0001 0001
-----------
  1100 1101
```

Overflow Flag

- This flag is set when the result of a signed operation goes beyond limits of the data type.
- The OF is set (1) when:
  - Two positive numbers are added and the result is negative (indicating overflow).
  - Two negative numbers are added and the result is positive (indicating overflow).

OF = 1

```
  0111 1111 (127)
+ 0000 0001 (1)
  ------------------
  1000 0000 (-128)
```

```
  1000 0001 (-127)
-  0000 0001 (1)
  ------------------
  0111 1110 (126)
```

OF = 0

```
  0000 0001 (1)
+ 0000 0010 (2)
  ----------------
  0000 0011 (3)
```

Carry Flag

- The CF is set (1) if there is a carry out from the MSB during addition.
- The CF is set (1) if there is a borrow from MSB in a subtraction operation.

```
  CF = 1
   1111 1111
+ 0000 0001
 ------------------
1 0000 0000
```

```
  CF = 1
   0000 0000
 - 0000 0001
 ------------------
(1)1111 1111
```

```
  CF = 0
   0000 1110
+ 0000 1111
 ------------------

0001 1101
```

```
  CF = 0
   0000 0010
 - 0000 0001
 ------------------
   0000 0001
```

Control Flags

- Dictate how the operations will be performed by the processor
- How certain instructions are executed
  - Interrupt flag (IF)
  - Trap Flag (TF)
  - Directional flag (DF)

Directional Flag

- Determines the direction in which string operations will be processed in memory
- DF = 0 (Forward Direction):
  - String operations will increment the index registers
  - Data is processed from lower memory addresses to higher memory addresses
- DF = 1 (Backward Direction):
  - String operations will decrement the index registers.
  - This means data is processed from higher memory addresses to lower memory addresses.
- CLD and STD

Trap Flag

- Enables single-step debugging by generating an interrupt after each instruction execution.
- **TF = 1 (Trap Mode)**:
  - When set, it allows a debugger to take control after each instruction, facilitating step-by-step execution.
- **TF = 0 (Normal Mode)**:
  - When cleared, the CPU executes instructions normally without generating interrupts after each instruction.

Interrupt Flag

- Controls whether the computer can respond to important signals (interrupts) from peripheral devices, like the keyboard or mouse
- **IF = 1** (Interrupts Enabled):
  - If you're typing in Word, the computer registers each keypress instantly because interrupts are enabled.
- **IF = 0** (Interrupts Disabled):
  - The CPU **ignores** outside signals temporarily.
  - During an update, interrupts might be disabled to ensure the update isn't interrupted by other tasks.

Practice Questions

1. Write an assembly code snippet to add two 8-bit numbers and store the result in the `AX` register. Check which flags are affected by this operation.
2. Perform a subtraction of `0x35` from `0x50` using the `SUB` instruction and determine the state of the Zero Flag (ZF) and Carry Flag (CF).
3. Multiply the contents of the `AX` register by the value in the `BX` register using the `MUL` instruction. What registers store the result, and what flags might be affected?
4. Using the `NEG` instruction, negate the value in the `DX` register. What happens to the Sign Flag (SF) and Overflow Flag (OF)?
5. Write code to add two values and include an immediate carry using the `ADC` instruction. Describe how the Carry Flag (CF) affects the result.